

Practical Exam in Analytics Application

Data Wrangling and Visualization

Instructions:

- **Replace the placeholders:** Replace `first_name` and `last_name` with your actual first and last names.
- **Run the code:** Click the "Run" button in the code chunk below to start the exam.
- **Complete the tasks:** Follow the instructions in the exam.
- **Save as PDF:** Once you've finished, save the document as a PDF using the format "LastName_FirstName.pdf" by printing the page from the browser [CTRL+P].

Student Information

▶ Run Code



```
1 first_name <- "Leilyn Mae"
2 last_name <- "Abellar"
3 cat(paste0(last_name, ", ", first_name, " [", Sys.time(), "]", sep=""))
```

Last Name, First Name [2024-09-23 23:35:17.422]

Assign the **Default** data to a data.frame: Use the assignment operator `<-` to assign the Default dataset to the dataset data.frame. This will create a copy of the Default data in your working environment.

Try glimpse on the dataset and print first 6 records from the data.frame.

▶ Run Code



```
1 df_data <- as.data.frame(Default)
2 print(str(df_data))
3 print(head(df_data))
```

```
'data.frame':  10000 obs. of  4 variables:
 $ default: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ student: Factor w/ 2 levels "No","Yes": 1 2 1 1 1 2 1 2 1 1 ...
 $ balance: num  730 817 1074 529 786 ...
 $ income : num  44362 12106 31767 35704 38463 ...
```

NULL

	default	student	balance	income
1	No	No	729.5265	44361.625
2	No	Yes	817.1804	12106.135
3	No	No	1073.5492	31767.139
4	No	No	529.2506	35704.494
5	No	No	785.6559	38463.496
6	No	Yes	919.5885	7491.559

Task 1: Calculate the median balance for each student status, but only for those with an income above the overall average income.

▶ Run Code



```

1  tb_data <- as_tibble(df_data)
2
3  avg_income <- mean(tb_data$income, na.rm = TRUE)
4
5  mbalance_data <- tb_data %>% filter(income > avg_income) %>% group_by
  (student) %>% summarize(median_balance = median(balance))
6  print(mbalance_data)

```

```

# A tibble: 1 × 2
  student median_balance
  <fct>         <dbl>
1 No           768.

```

Task 2: Consider students from the dataset. Group them by default status, and for each group, calculate the number students and the average income.

▶ Run Code



```

1  tb_data %>% filter(student == "Yes") %>% group_by(default) %>% summarize
  (number_of_students = n(), avg_credit_limit = (mean(income, na.rm = TRUE)))

```

```

# A tibble: 2 × 3
  default number_of_students avg_credit_limit
  <fct>         <int>         <dbl>
1 No           2817         17937.
2 Yes           127         18244.

```

Task 3: Create a new column `income_level` categorizing income into 'Low' and 'High' with the mean as basis, where high income is above average. Then, determine how many records belong to each.

▶ Run Code



```

1  avg_income <- mean(tb_data$income, na.rm = TRUE)
2
3  mutated_data <- tb_data %>% mutate(income_level = case_when(
4    income > avg_income ~ "High",
5    TRUE ~ "Low"))
6
7  gs_data <- mutated_data %>% group_by(income_level) %>% summarise(count = n
  ())
8  print(gs_data)

```

```

# A tibble: 2 × 2
  income_level count
  <chr>         <int>
1 High           5264
2 Low            4736

```

Task 4: Based on your previous output, determine how many defaulted for each combination of student status and `income_level`.

Run Code



```

1  avg_income <- mean(tb_data$income, na.rm = TRUE)
2
3  mutated_data <- tb_data %>%
4    mutate(income_level = case_when(
5      income > avg_income ~ "High",
6      TRUE ~ "Low"
7    ))
8
9  gs_data <- mutated_data %>% group_by(student, income_level, default) %>%
  summarise(defaulted_records = sum(default == "Yes"), .groups = 'drop')
10 filtered_default <- gs_data %>% filter(default == "Yes")
11 print(filtered_default)

```

A tibble: 3 × 4

	student	income_level	default	defaulted_records
	<fct>	<chr>	<fct>	<int>
1	No	High	Yes	158
2	No	Low	Yes	48
3	Yes	Low	Yes	127

Task 5: Find the top 5 customers with the highest balance who have not defaulted, and display their student status, balance, and income.

Run Code



```

1  not_defaulted <- tb_data %>% filter(default == "No")
2
3  top_customers <- not_defaulted %>% arrange(desc(balance)) %>% head(5)
4  result <- top_customers %>% select(student, balance, income)
5  print(result)

```

A tibble: 5 × 3

	student	balance	income
	<fct>	<dbl>	<dbl>
1	No	2391.	50303.
2	Yes	2388.	7832.
3	Yes	2370.	24252.
4	Yes	2309.	19110.
5	Yes	2216.	24737.

Task 6: Create a summary table showing the count of customers, average balance, and median income for each combination of student and default status.

Run Code



```

1  summary_table <- tb_data %>% group_by(student, default) %>% summarise
  (count_customers = n(), average_balance = mean(balance, na.rm = TRUE),
  median_income = median(income, na.rm = TRUE), .groups = 'drop')
2  print(summary_table)

```

```
# A tibble: 4 × 5
```

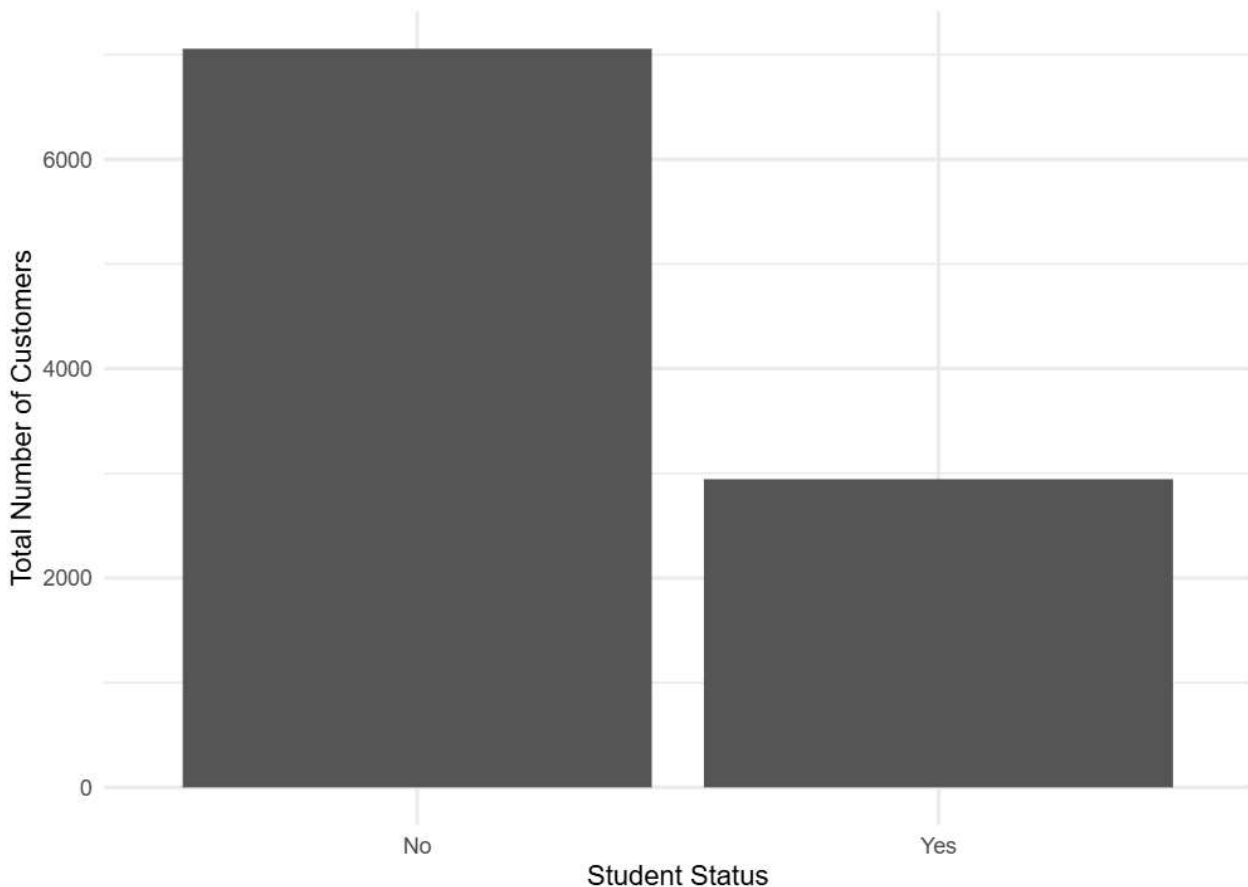
	student	default	count_customers	average_balance	median_income
	<fct>	<fct>	<int>	<dbl>	<dbl>
1	No	No	6850	745.	39886.
2	No	Yes	206	1678.	40281.
3	Yes	No	2817	948.	17992.
4	Yes	Yes	127	1860.	18021.

Task 7: Generate bar chart using ggplot summarizing number of customers by student status.

▶ Run Code



```
1 summary_data <- tb_data %>% group_by(student) %>% summarise(count = n())
2
3 ggplot(summary_data, aes(x = student, y = count)) +
4   geom_bar(stat = "identity") +
5   labs(x = "Student Status",
6        y = "Total Number of Customers") +
7   theme_minimal()
```



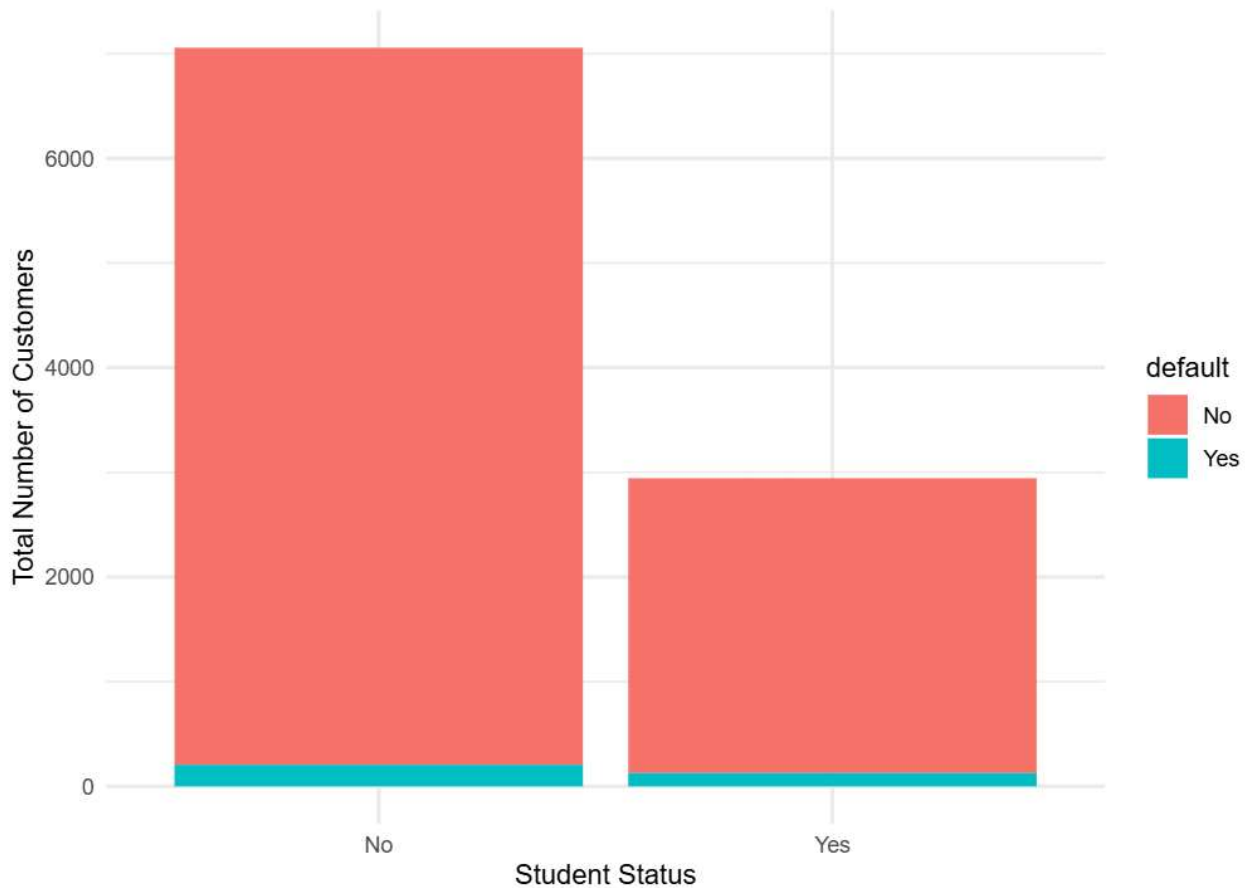
Task 8: Generate bar chart using ggplot summarizing number of customers by student status filled by default status.

▶ Run Code



```
1 summary_data <- tb_data %>% group_by(student, default) %>% summarise(count
  = n(), .groups = 'drop')
```

```
3  ggplot(summary_data, aes(x = student, y = count, fill = default)) +  
4    geom_bar(stat = "identity") +  
5    labs(x = "Student Status",  
6         y = "Total Number of Customers") +  
7    theme_minimal()
```

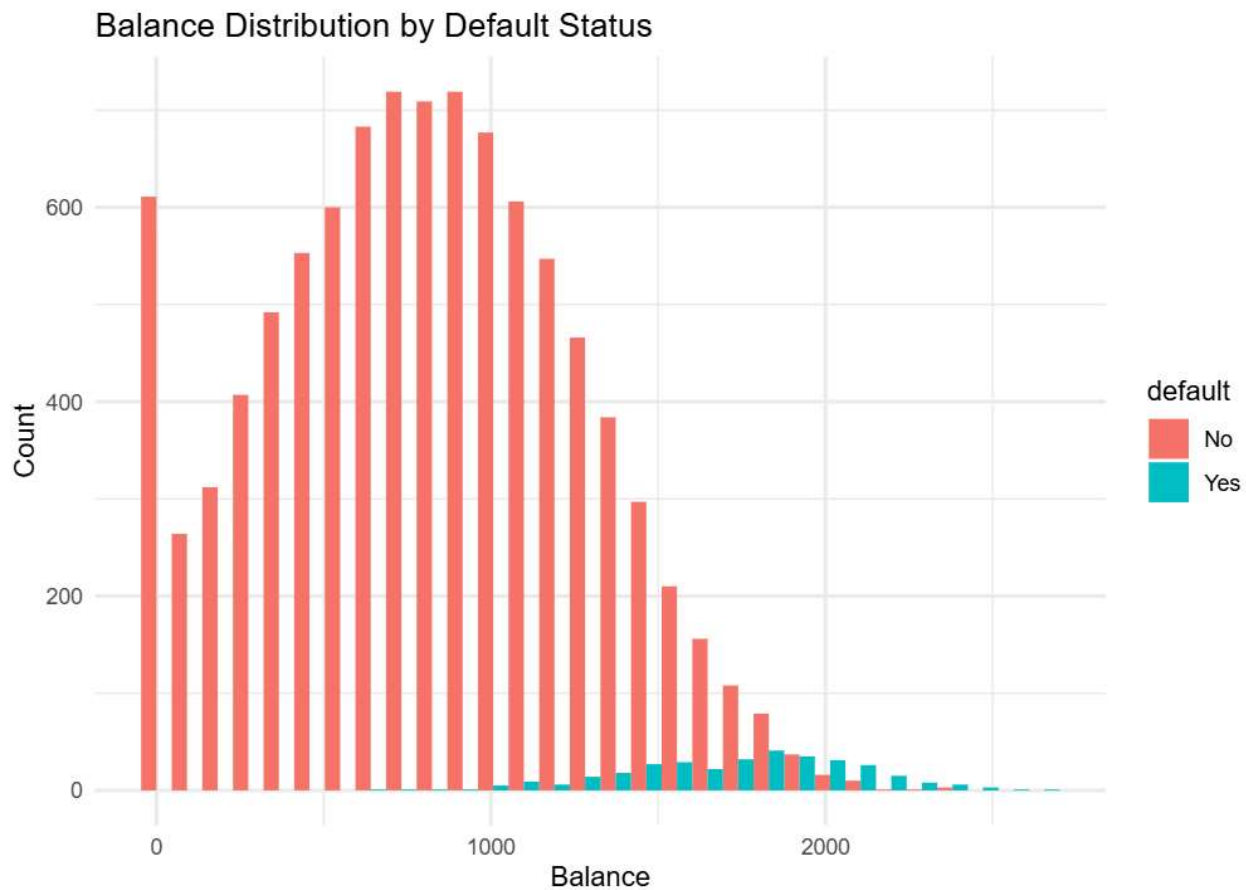


Task 9: Create a histogram of the balance variable, colored by default status. Add appropriate labels and a title.

▶ Run Code



```
1  ggplot(tb_data, aes(x = balance, fill = default)) +  
2    geom_histogram(position = "dodge", bins = 30) +  
3    labs(title = "Balance Distribution by Default Status",  
4         x = "Balance",  
5         y = "Count") +  
6    theme_minimal()
```

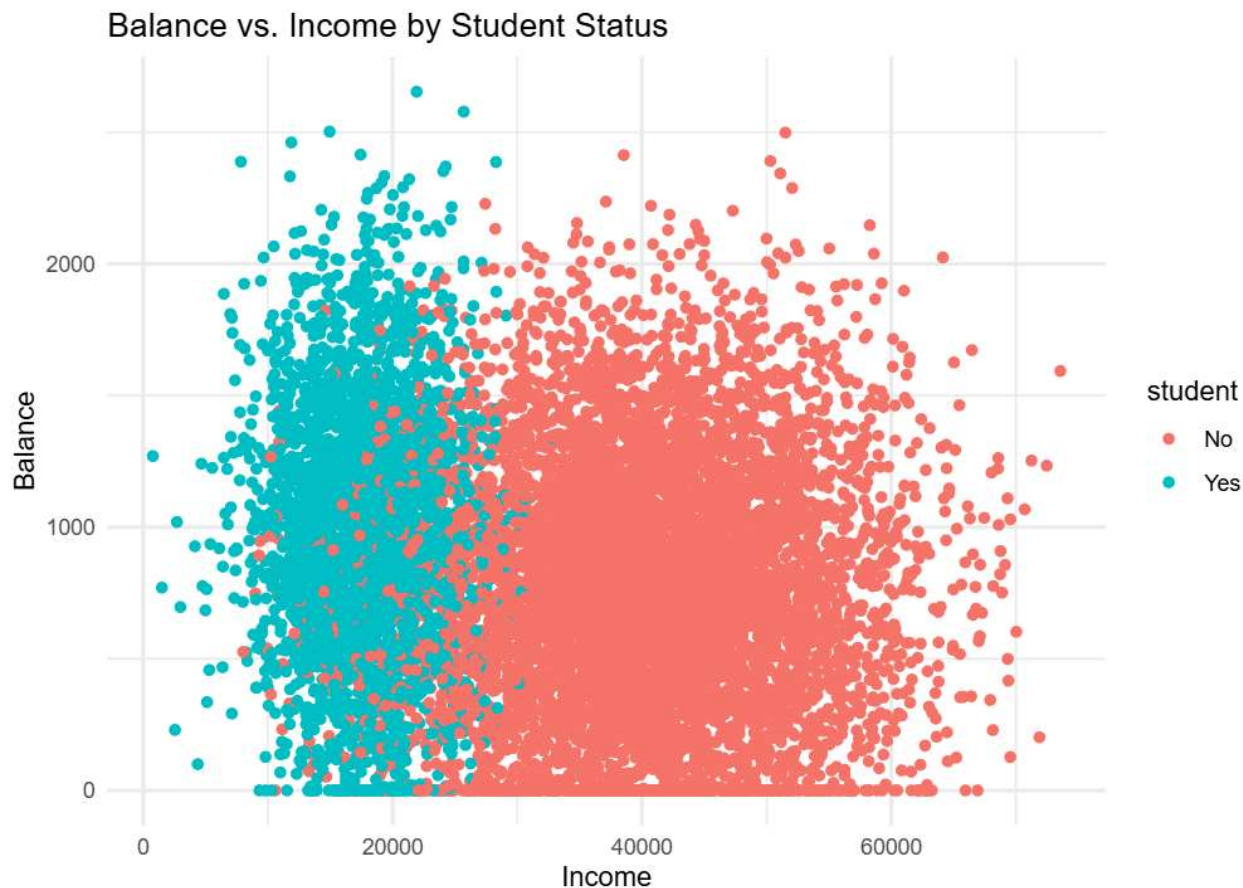


Task 10: Create a scatterplot of balance vs. income, with the points colored by student status. Include a legend and suitable axis labels.

Run Code



```
1 ggplot(tb_data, aes(x = income, y = balance, color = student)) +  
2   geom_point(alpha = 1) +  
3   labs(title = "Balance vs. Income by Student Status",  
4         x = "Income",  
5         y = "Balance",  
6         color = "student") +  
7   theme_minimal()
```

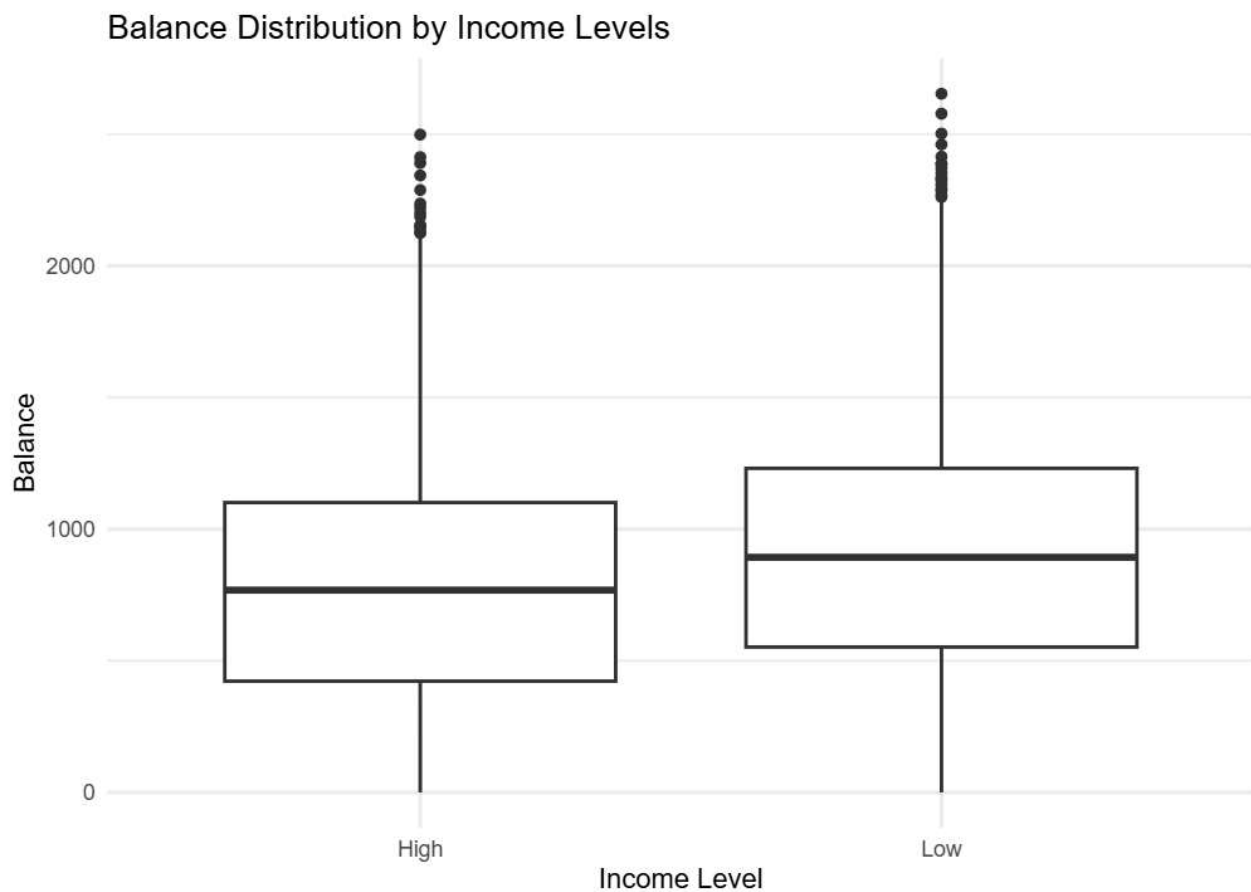


Task 11: Create boxplots to visualize the distribution of balance across different income_level categories (created in the previous dplyr tasks).

Run Code



```
1 avg_income <- mean(tb_data$income, na.rm = TRUE)
2
3 tb_data <- tb_data %>% mutate(income_level = ifelse(income > avg_income,
4 "High", "Low"))
5
6 ggplot(tb_data, aes(x = income_level, y = balance)) +
7   geom_boxplot() +
8   labs(title = "Balance Distribution by Income Levels",
9         x = "Income Level",
10        y = "Balance") +
11   theme_minimal()
```

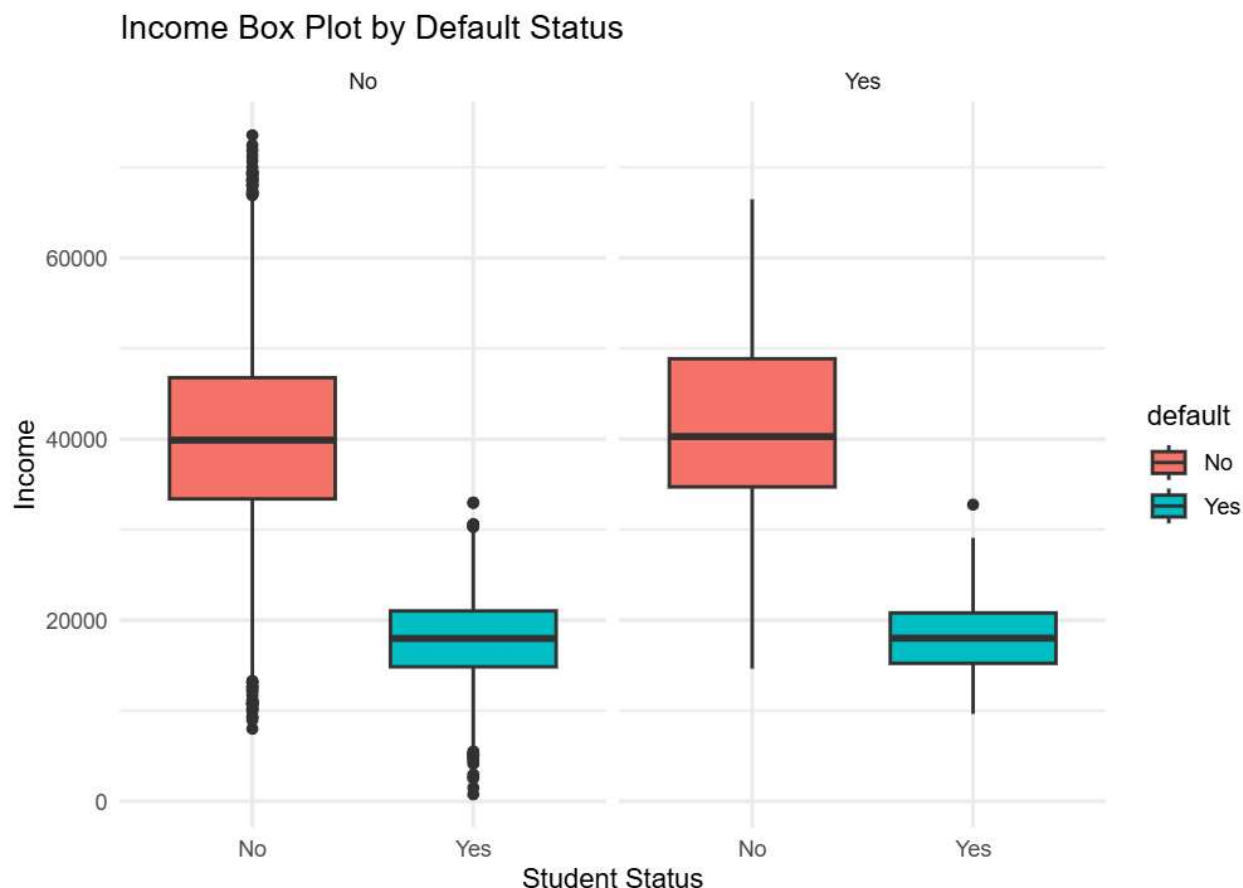


Task 12: Generate a box plot of income by student status, faceted by default status. Ensure the plots have clear labels and a title.

▶ Run Code



```
1 ggplot(tb_data, aes(x = student, y = income, fill = student)) +  
2   geom_boxplot() +  
3   facet_wrap(~ default) +  
4   labs(title = "Income Box Plot by Default Status",  
5         x = "Student Status",  
6         y = "Income",  
7         fill = "default") +  
8   theme_minimal()
```



For the next three tasks consider new data.frame.

▶ Run Code



```

1 # Define the years and months
2 years <- c(2022, 2023)
3 months <- month.name
4
5 # Create a data frame with all combinations of years and months
6 df <- expand.grid(Year = years, Month = months)
7
8 # Generate random sales data for Product A and Product B
9 set.seed(123)
10 df$Product_A <- round(runif(nrow(df), 50, 200))
11 df$Product_B <- round(runif(nrow(df), 30, 150))
12
13 # Reorder the months chronologically
14 df$Month <- factor(df$Month, levels = months)
15 df <- df[order(df$Year, df$Month), ]
16
17 # Create the As_of_Month column with the desired format
18 df$As_of_Month <- paste(df$Year, sprintf("%02d", match(df$Month, month.
19   name)), "15", sep = "-")
20 # Print the data frame

```

	21	print(df)				
	Year	Month	Product_A	Product_B	As_of_Month	
1	2022	January	93	109	2022-01-15	
3	2022	February	111	95	2022-02-15	
5	2022	March	191	65	2022-03-15	
7	2022	April	129	146	2022-04-15	
9	2022	May	133	113	2022-05-15	
11	2022	June	194	33	2022-06-15	
13	2022	July	152	121	2022-07-15	
15	2022	August	65	68	2022-08-15	
17	2022	September	87	47	2022-09-15	
19	2022	October	99	80	2022-10-15	
21	2022	November	183	48	2022-11-15	
23	2022	December	146	58	2022-12-15	
2	2023	January	168	115	2023-01-15	
4	2023	February	182	101	2023-02-15	
6	2023	March	57	48	2023-03-15	
8	2023	April	184	138	2023-04-15	
10	2023	May	118	125	2023-05-15	
12	2023	June	118	87	2023-06-15	
14	2023	July	136	56	2023-07-15	
16	2023	August	185	58	2023-08-15	
18	2023	September	56	80	2023-09-15	
20	2023	October	193	74	2023-10-15	
22	2023	November	154	47	2023-11-15	
24	2023	December	199	86	2023-12-15	

Task 13: Reshape the data using pivot_longer from tidyr. Combine sales data for the 2 products into a new column Product.

 Run Code

```
1 data_long <- df %>%
2   pivot_longer(cols = starts_with("Product"),
3                 names_to = "Product",
4                 values_to = "Sales")
5 print(data_long)
```

A tibble: 48 × 5

	Year	Month	As_of_Month	Product	Sales
	<dbl>	<fct>	<chr>	<chr>	<dbl>
1	2022	January	2022-01-15	Product_A	93
2	2022	January	2022-01-15	Product_B	109
3	2022	February	2022-02-15	Product_A	111
4	2022	February	2022-02-15	Product_B	95
5	2022	March	2022-03-15	Product_A	191

6	2022	March	2022-03-15	Product_B	65
7	2022	April	2022-04-15	Product_A	129
8	2022	April	2022-04-15	Product_B	146
9	2022	May	2022-05-15	Product_A	133
10	2022	May	2022-05-15	Product_B	113

i 38 more rows

Task 14: Using the new structured data, generate bar chart for the sales report for the year 2023.

Run Code



```

1 data_long <- df %>% pivot_longer(cols = starts_with("Product"), names_to =
  "Product", values_to = "Sales")
2
3 data_2023 <- data_long %>% filter(Year == 2023)
4
5 ggplot(data_2023, aes(x = Month, y = Sales, fill = Product)) +
6   geom_bar(stat = "identity", position = "dodge") +
7   labs(title = "2023 Sales by Product and Month",
8     x = "Month",
9     y = "Sales",
10    fill = "Product") +
11   theme_minimal()

```

**Task 15:** Using the new structured data, generate bar chart for the sales report for the year 2023 and overlay line chart for 2022 data.

Run Code



```
1 data_long <- df %>% pivot_longer(cols = starts_with("Product"), names_to =  
  "Product", values_to = "Sales")  
2  
3 data_2022 <- data_long %>% filter(Year == 2022) %>% group_by(Month) %>%  
  summarize(Sales = sum(Sales))  
4  
5 data_2023 <- data_long %>% filter(Year == 2023) %>% group_by(Month) %>%  
  summarize(Sales = sum(Sales))  
6  
7 ggplot() +  
8   geom_bar(data = data_2023, aes(x = Month, y = Sales),  
9           stat = "identity", fill = "blue") +  
10  geom_point(data = data_2022, aes(x = Month, y = Sales),  
11           color = "orange", size = 1) +  
12  labs(x = "Month",  
13       y = "Sales") +  
14  theme_minimal()
```

